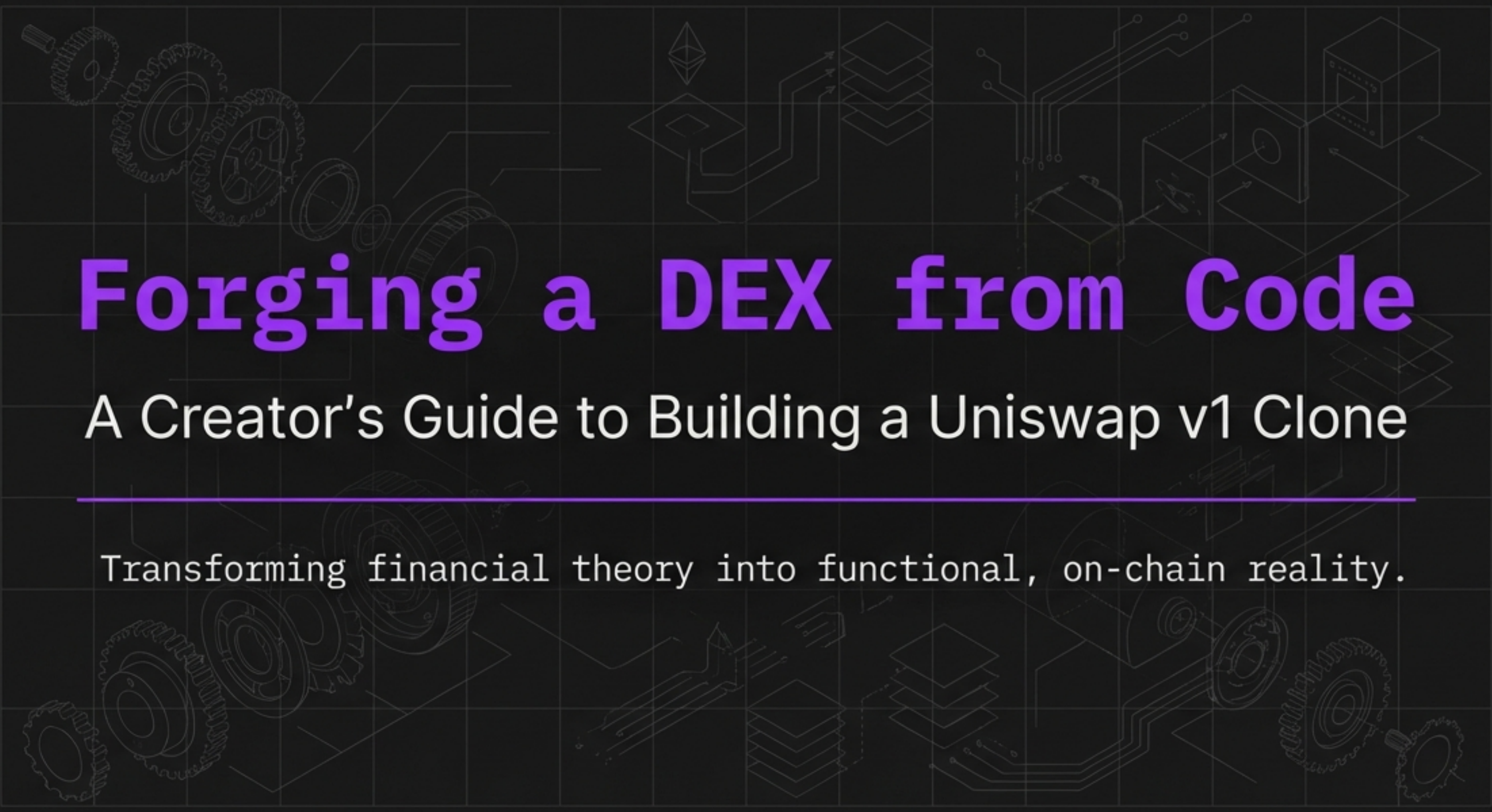


The background features a dark gray grid with faint, light gray technical drawings. These drawings include various mechanical components like gears and cogs, as well as abstract geometric shapes and circuit-like lines, creating a complex, engineering-inspired aesthetic.

Forging a DEX from Code

A Creator's Guide to Building a Uniswap v1 Clone

Transforming financial theory into functional, on-chain reality.

The Blueprint: Our Mission's Core Requirements

We are building a decentralised exchange (DEX) from first principles. Our clone of Uniswap v1 will serve a specific purpose with four key functional requirements.



1. Core Functionality

Enable swaps between a native asset (ETH) and an ERC-20 token (TOKEN).



2. Economic Model

Implement a 1% fee on all swaps, creating a sustainable protocol.



3. Liquidity Provision

Issue unique Liquidity Provider (LP) tokens to users who deposit assets, representing their share of the pool.



4. Capital Efficiency

Allow LPs to burn their tokens at any time to reclaim their proportional share of ETH and TOKEN from the liquidity pool.

Preparing the Workspace: Environment and Tools



Foundry Setup (The Forge)

We will use Foundry, a powerful toolkit for Ethereum application development, to build and test our smart contracts.

```
# Initialise the project
forge init foundry-app
cd foundry-app

# Install OpenZeppelin for ERC-20 standard
forge install OpenZeppelin/openzeppelin-contracts

# Configure Import remappings
forge remappings > remappings.txt
```



Environment Variables (The Keys)

Your `env` file holds the necessary credentials for deploying and verifying on the Sepolia testnet.

```
PRIVATE_KEY="..." # Your wallet's private key (testnet funds only)
RPC_URL="..."      # Your QuickNode Sepolia HTTP endpoint
ETHERSCAN_API_KEY="..." # Your Etherscan API key for verification
```


Forging the Raw Material: A Simple ERC-20 Token

Before building the exchange, we need a token to trade against ETH. We'll create a basic ERC-20 contract (Token.sol) that mints a starting supply to the contract deployer.

Token.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.25;
3
4 import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5
6 // A standard ERC-20 token contract.
7 contract Token is ERC20 {
8     // The constructor is called only once on deployment.
9     constructor() ERC20("Token", "TKN") {
10         // Mint 1 million tokens to the creator of the contract.
11         // The total supply is fixed from this point.
12         _mint(msg.sender, 1000000 * 10 ** decimals());
13     }
14 }
```

The constructor is called only once on deployment.

Mint 1 million tokens to the creator of the contract. The total supply is fixed from this point.

This self-contained contract provides the asset we need for our liquidity pool.

Architecting the Exchange: The Contract Skeleton

The `Exchange.sol` contract is the heart of our DEX. It manages liquidity pools and facilitates swaps. Crucially, it also inherits from ERC20, as it is responsible for minting and managing the LP tokens.

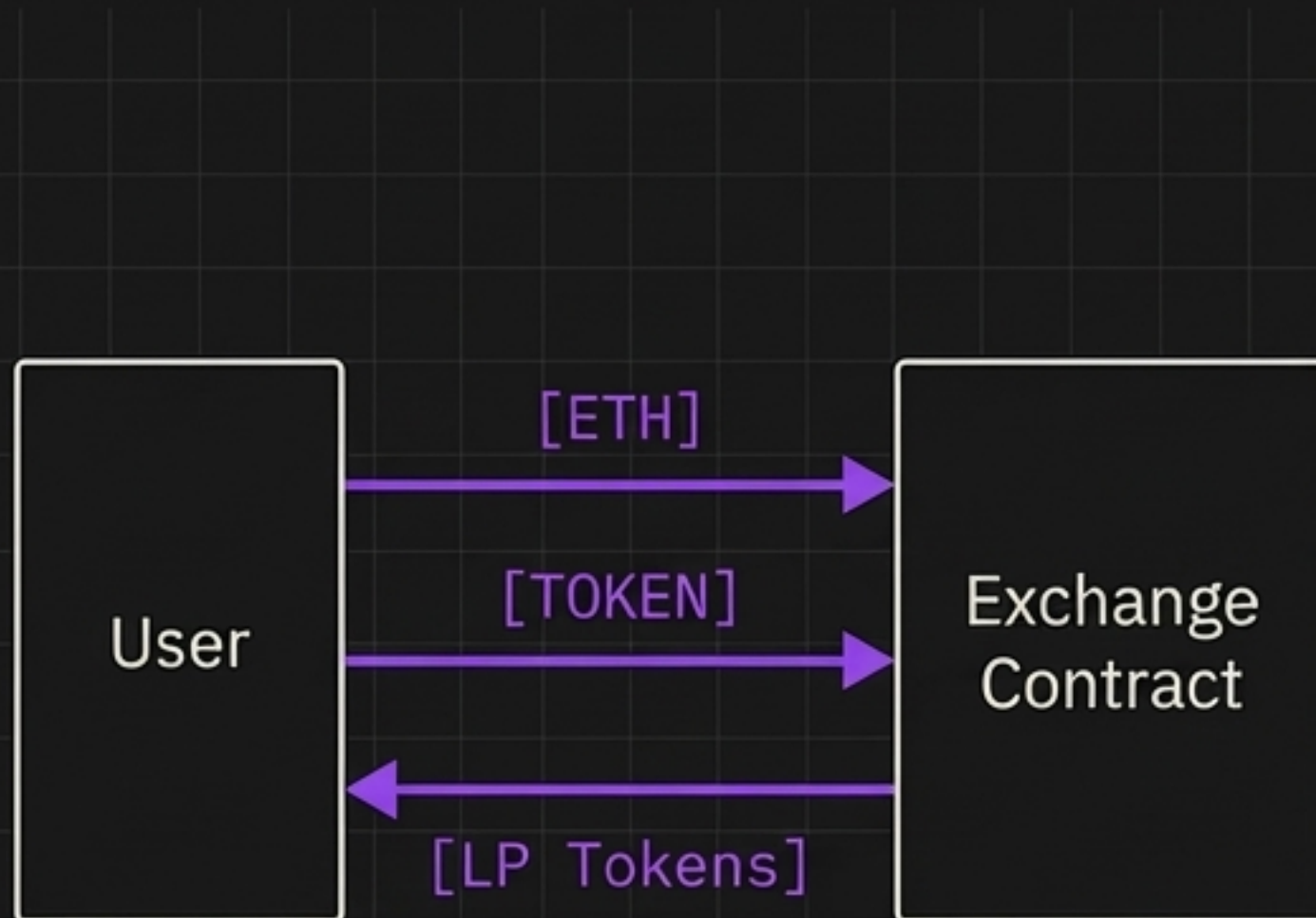
```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.25;
3
4  import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5
6  // Inherits from ERC20 to become an LP token contract.
7  contract Exchange is ERC20 {
8      address public tokenAddress;
9
10     // The constructor locks in the ERC-20 token for this exchange.
11     constructor(address token) ERC20("ETH TOKEN LP Token", "lpETHTOKEN") {
12         require(token != address(0), "Token address passed is a null address");
13         tokenAddress = token;
14     }
15
16     // A simple view function to check the exchange's token balance.
17     function getReserve() public view returns (uint256) {
18         return ERC20(tokenAddress).balanceOf(address(this));
19     }
20 }
```

Inherits from ERC20 to become an LP token contract.

The constructor locks in the specific ERC-20 token for this exchange instance.

The Lifeblood of the Exchange: Adding Liquidity

A DEX is useless without assets to trade. The `addLiquidity` function allows users to **deposit a proportional amount of ETH and TOKEN**, receiving LP tokens in return. The logic adapts based on whether the pool is empty.



```
function addLiquidity(uint256 amountOfToken) public payable returns (uint256) {  
    // 1. Handle the first deposit (empty reserve)  
    if (getReserve() == 0) {  
        // Transfer tokens from user to this contract.  
        ERC20(tokenAddress).transferFrom(msg.sender, address(this), amountOfToken);  
  
        // The amount of LP tokens minted is equal to the ETH deposited.  
        uint256 lpTokensToMint = address(this).balance;  
        _mint(msg.sender, lpTokensToMint);  
        return lpTokensToMint;  
    } else {  
        // 2. For existing reserves, calculate the required token amount.  
        uint256 ethReserve = address(this).balance - msg.value;  
        uint256 tokenReserve = getReserve();  
        uint256 minTokenAmount = (msg.value * tokenReserve) / ethReserve;  
  
        require(amountOfToken >= minTokenAmount, "Insufficient amount of tokens");  
  
        // Transfer the required amount of tokens from the user.  
        ERC20(tokenAddress).transferFrom(msg.sender, address(this), minTokenAmount);  
  
        // Calculate LP tokens to mint based on the user's share.  
        uint256 lpTokensToMint = (totalSupply() * msg.value) / ethReserve;  
        _mint(msg.sender, lpTokensToMint);  
        return lpTokensToMint;  
    }  
}
```

Handles the initial deposit, setting the first exchange rate.

Maintains the exchange rate for all subsequent deposits.

Reclaiming Capital: The Logic of Removing Liquidity

Liquidity providers can exit their position at any time. The `removeLiquidity` function calculates their share of the pool based on the amount of LP tokens they hold, **burns** those tokens, and transfers the corresponding ETH and TOKEN back to them.



```
function removeLiquidity(uint256 amountOfLPTokens) public returns (uint256, uint256) {
    require(amountOfLPTokens > 0, "Amount must be greater than 0");

    uint256 ethReserve = address(this).balance;
    uint256 lpTotalSupply = totalSupply();

    // Calculate the user's proportional share of ETH and TOKEN.
    uint256 ethToReturn = (ethReserve * amountOfLPTokens) / lpTotalSupply;
    uint256 tokenToReturn = (getReserve() * amountOfLPTokens) / lpTotal;
    // Burn the user's LP tokens.
    _burn(msg.sender, amountOfLPTokens);

    // Transfer the underlying assets back to the user.
    payable(msg.sender).transfer(ethToReturn);
    ERC20(tokenAddress).transfer(msg.sender, tokenToReturn);

    return (ethToReturn, tokenToReturn);
}
```

Calculates the user's proportional share of the reserves.

Burns the LP tokens and returns the underlying assets.

The Engine of Exchange: Calculating the Swap

At the core of our AMM is the constant product formula: $x * y = k$. This function, `getOutputAmountFromSwap`, calculates how many output tokens a user receives for a given input amount, while maintaining the constant product and accounting for a 1% protocol fee.

$$\text{Core Formula: } (x + \Delta x) * (y - \Delta y) = k$$

Our Implementation: We calculate Δy (the output amount).

```
function getOutputAmountFromSwap(  
    uint256 inputAmount,  
    uint256 inputReserve,  
    uint256 outputReserve  
) public pure returns (uint256) {  
    require(inputReserve > 0 && outputReserve > 0, "Reserves must be > 0");  
  
    // Apply a 1% fee (99% of input is used for the swap).  
    uint256 inputAmountWithFee = inputAmount * 99;   
  
    // Calculate the numerator and denominator for the output amount.  
    uint256 numerator = inputAmountWithFee * outputReserve;  
    uint256 denominator = (inputReserve * 100) + inputAmountWithFee;  
  
    return numerator / denominator;  
}
```

The 1% protocol fee is applied here by using 99% of the input for the calculation.

Opening the Gates: Executing the Swaps

Concept: These two functions are the primary interface for traders. They take user input, call `getOutputAmountFromSwap` to determine the outcome, and then perform the necessary token transfers.

ETH to Token Swap

```
function ethToTokenSwap(uint256 minTokensToReceive) public payable {
    uint256 tokenReserve = getReserve();
    uint256 tokensToReceive = getOutputAmountFromSwap(
        msg.value,
        address(this).balance - msg.value, ← tokenReserve
    );

    require(tokensToReceive >= minTokensToReceive, "Slippage tolerance failed");

    ERC20(tokenAddress).transfer(msg.sender, tokensToReceive);
}
```

Important: The input reserve is the contract's balance *before* the current deposit is added.

Token to ETH Swap

```
function tokenToEthSwap(uint256 tokensToSwap, uint256 minEthToReceive) public {
    uint256 tokenReserve = getReserve();
    uint256 ethToReceive = getOutputAmountFromSwap(
        tokensToSwap,
        tokenReserve,
        address(this).balance
    );

    require(ethToReceive >= minEthToReceive, "Slippage tolerance failed");

    { ERC20(tokenAddress).transferFrom(msg.sender, address(this), tokensToSwap);
      payable(msg.sender).transfer(ethToReceive); } ← The contract pulls tokens from the user, then sends ETH back.
}
```


A Moment of Truth: Deployment to the Blockchain



With the contracts complete, we deploy them to the Sepolia testnet using Foundry. The `--verify` flag also publishes the source code to Etherscan, making our DEX transparent and publicly auditable.

Step 1: Deploy the Token Contract

First, we deploy `Token.sol`. We need to capture its address for the next step.

```
forge create --rpc-url $RPC_URL --private-key $PRIVATE_KEY \
  --etherscan-api-key $ETHERSCAN_API_KEY --verify \
  src/Token.sol:Token
```

Step 2: Deploy the Exchange Contract

Now, we deploy `Exchange.sol`, passing the newly created token's address to its constructor.

```
forge create --rpc-url $RPC_URL --private-key $PRIVATE_KEY \
  --constructor-args <address_of_token_contract> \
  --etherscan-api-key $ETHERSCAN_API_KEY --verify \
  src/Exchange.sol:Exchange
```


The Proving Ground: Adding Liquidity via Etherscan

We will now interact directly with our deployed contracts on Sepolia Etherscan to verify their functionality. The first test is to provide the initial liquidity.

Step 1: Approve Token Spending

On the `Token` contract's "Write Contract" tab, call the `approve` function.

Token.sol

approve

spender	The address of your deployed `Exchange` contract.
amount	"1000000000000000000000000"

This is a standard ERC-20 flow. You must grant permission before the Exchange contract can transfer tokens on your behalf.

Step 2: Add Liquidity

On the `Exchange` contract's "Write Contract" tab, call the `addLiquidity` function.

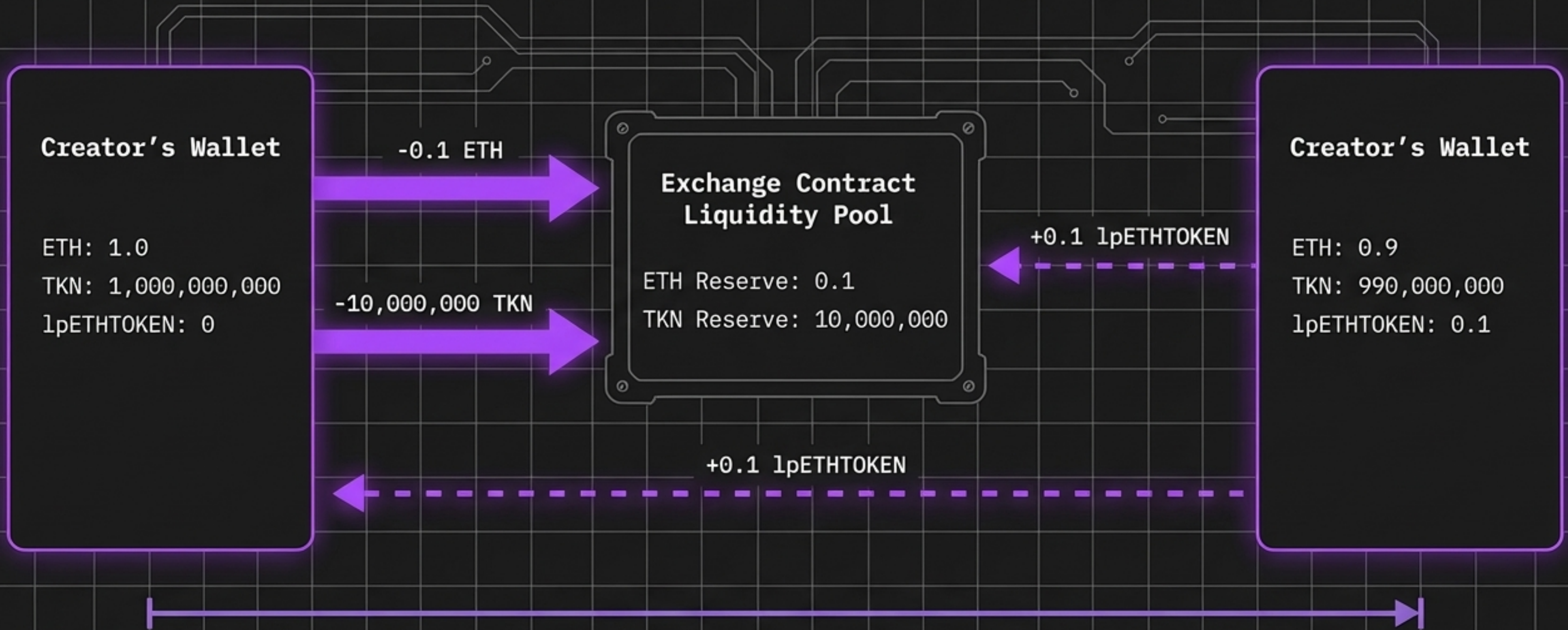
Exchange.sol

addLiquidity

payableAmount (ETH)	"0.1"
amountOfToken	"100000000"

After the transaction confirms, you will see TOKENs leave your wallet and new `1pETHTOKEN` tokens minted to your wallet.

Visualising the First Deposit



The Inaugural Trade: Executing Swaps on Etherscan

Now that our exchange has liquidity, let's perform both an ETH-to-Token and a Token-to-ETH swap.

Swap 1: ETH → Token (`ethToTokenSwap`)

- Action: Call `ethToTokenSwap` on the `Exchange` contract.
- Parameters:
 - `payableAmount` (ETH): `0.01`
 - `minTokensToReceive`: `0` (for testing purposes)
- Result: You send 0.01 ETH and receive a calculated amount of TOKEN back.

Swap 2: Token → ETH (`tokenToEthSwap`)

- Action: Call `tokenToEthSwap` on the `Exchange` contract.
- Parameters:
 - `tokensToSwap`: `10000`
 - `minEthToReceive`: `0`
- Result: You send 10,000 TOKEN and receive a calculated amount of ETH back.



In a real front-end, `minTokensToReceive` is crucial for protecting users from slippage. It's calculated based on the expected output minus a user-agreed slippage percentage (e.g., 1%). Setting it to zero is only for simple testing.

Completing the Cycle: Removing Liquidity

The final test is to ensure that liquidity providers can successfully withdraw their assets. This confirms the entire round-trip functionality of our DEX.

Step 1: Check LP Token Balance

Read Contract

``balanceOf``

Data

``account``: Your wallet address.

Result:
This shows your exact balance of ``1pETHTOKEN``. For the example, this is ``1000000000000000000``.

Step 2: Remove a Portion of Liquidity

Write Contract

``removeLiquidity``

Data

``amountOfLPTokens``:
``5000000000000000000``

Result: The specified LP tokens are burned from your wallet, and a proportional amount of ETH and TOKEN are transferred back to you, successfully completing the process.

The Completed Creation: A Functional On-Chain Exchange

From a blank file to a live application on the blockchain, you have successfully forged a decentralised exchange. You have engineered a system that:

- **Accepts Capital:** Manages liquidity deposits via the `addLiquidity` function.
- **Issues Shares:** Mints proportional LP tokens to represent ownership.
- **Calculates Price:** Uses a constant product formula (`getOutputAmountFromSwap`) to determine trade outcomes.
- **Executes Trades:** Facilitates swaps between ETH and an ERC-20 token.
- **Returns Capital:** Allows LPs to burn shares and reclaim their assets.

